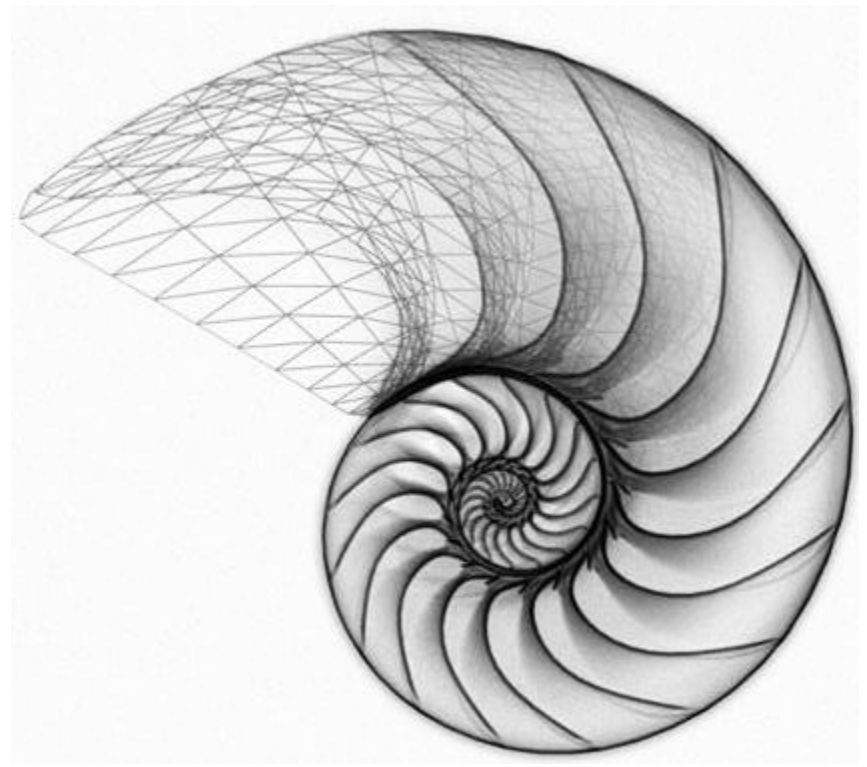




Lecture 10

Recursion

CSE225: Data Structures and Algorithms

A Look Back at Functions

```
#include <stdio.h>
#include <math.h>
double distance(double x1, double y1, double x2, double y2)
{
    double dist;
    dist = sqrt((x1-x2)*(x1-x2)+(y1-y2)*(y1-y2));
    return dist;
}
int main()
{
    double milage = distance(0, 0, 3, 4);
    printf("Milage: %lf\n", milage);
    return 0;
}
```

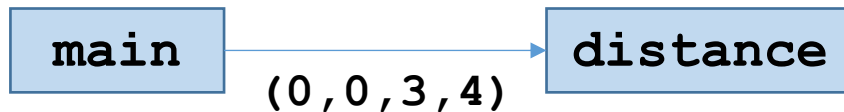
A Look Back at Functions

```
#include <stdio.h>
#include <math.h>
double distance(double x1, double y1, double x2, double y2)
{
    double dist;
    dist = sqrt((x1-x2)*(x1-x2)+(y1-y2)*(y1-y2));
    return dist;
}
int main()
{
    double milage = distance(0, 0, 3, 4);
    printf("Milage: %lf\n", milage);
    return 0;
}
```

main

A Look Back at Functions

```
#include <stdio.h>
#include <math.h>
double distance(double x1, double y1, double x2, double y2)
{
    double dist;
    dist = sqrt((x1-x2)*(x1-x2)+(y1-y2)*(y1-y2));
    return dist;
}
int main()
{
    double milage = distance(0, 0, 3, 4);
    printf("Milage: %lf\n", milage);
    return 0;
}
```



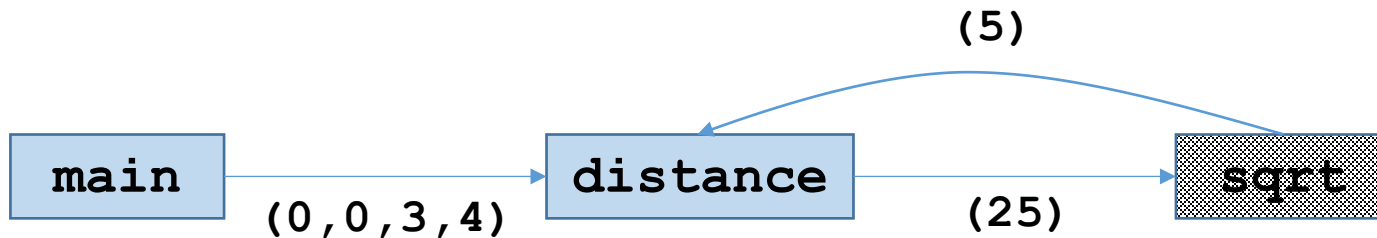
A Look Back at Functions

```
#include <stdio.h>
#include <math.h>
double distance(double x1, double y1, double x2, double y2)
{
    double dist;
    dist = sqrt((x1-x2)*(x1-x2)+(y1-y2)*(y1-y2));
    return dist;
}
int main()
{
    double milage = distance(0, 0, 3, 4);
    printf("Milage: %lf\n", milage);
    return 0;
}
```



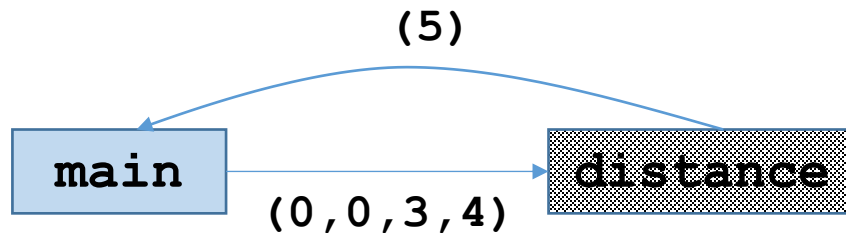
A Look Back at Functions

```
#include <stdio.h>
#include <math.h>
double distance(double x1, double y1, double x2, double y2)
{
    double dist;
    dist = sqrt((x1-x2)*(x1-x2)+(y1-y2)*(y1-y2));
    return dist;
}
int main()
{
    double milage = distance(0, 0, 3, 4);
    printf("Milage: %lf\n", milage);
    return 0;
}
```



A Look Back at Functions

```
#include <stdio.h>
#include <math.h>
double distance(double x1, double y1, double x2, double y2)
{
    double dist;
    dist = sqrt((x1-x2)*(x1-x2)+(y1-y2)*(y1-y2));
    return dist;
}
int main()
{
    double milage = distance(0, 0, 3, 4);
    printf("Milage: %lf\n", milage);
    return 0;
}
```



Recursion

- **Recursion** is a technique that solves a problem by solving a smaller problem of the same type.
- Recursion can be implemented using a **recursive function** (a function invoking itself, either directly or indirectly).
- Recursion can be used as an alternative to iteration.
- It is an important and powerful tool in problem solving and programming.
- It is a programming technique that naturally implements the divide-and-conquer problem solving methodology.



Factorial – Iterative Definition

$$n! = n * (n-1) * (n-2) * \dots * 2 * 1$$

for any integer $n > 0$

$$0! = 1$$

Iterative Definition in C:

```
fval = 1;  
for (i = n; i >= 1; i--)  
    fval = fval * i;
```

Factorial – Recursive Definition

- To define ***n!*** recursively, ***n!*** must be defined in terms of the factorial of a smaller number.
- Observation (problem size is reduced):
$$n! = n * (n-1)!$$
- Base case: $0! = 1$
- We can reach the base case, by subtracting 1 from n if n is a positive integer.

Recursive Definition:

$$n! = 1$$

$$\text{if } n = 0$$

$$n! = n * (n-1)!$$

$$\text{if } n > 0$$

The Nature of Recursion

1. One or more simple cases of the problem (called the *stopping cases* or *base case*) have a simple non-recursive solution.
2. The other cases (general cases) of the problem can be reduced (*using recursion*) to problems that are closer to stopping cases.
3. Eventually the problem can be reduced to base cases only, which are relatively easy to solve.

In general:

if (*base case*)

solve it

else

reduce the problem using recursion // general case

Factorial of 4 (Recursive)

Calculate 4!

$$4! = 4 * (4-1)!$$

$$4! = 4 * 3!$$

$$4! = 4 * _ _$$

Factorial of 4 (Recursive)

Calculate 4!

$$4! = 4 * (4-1)!$$

$$4! = 4 * 3!$$

$$4! = 4 * _ _$$

Calculate 3!

$$3! = 3 * (3-1)!$$

$$3! = 3 * 2!$$

$$3! = 3 * _ _$$

Factorial of 4 (Recursive)

Calculate 4!

$$4! = 4 * (4-1)!$$

$$4! = 4 * 3!$$

$$4! = 4 * _ _$$

Calculate 3!

$$3! = 3 * (3-1)!$$

$$3! = 3 * 2!$$

$$3! = 3 * _ _$$

Calculate 2!

$$2! = 2 * (2-1)!$$

$$2! = 2 * 1!$$

$$2! = 2 * _ _$$

Factorial of 4 (Recursive)

Calculate 4!

$$4! = 4 * (4-1)!$$

$$4! = 4 * 3!$$

$$4! = 4 * _ _ \text{Calculate 3!}$$

$$3! = 3 * (3-1)!$$

$$3! = 3 * 2!$$

$$3! = 3 * _ _ \text{Calculate 2!}$$

$$2! = 2 * (2-1)!$$

$$2! = 2 * 1!$$

$$2! = 2 * _ _ \text{Calculate 1!}$$

$$1! = 1 * (1-1)!$$

$$1! = 1 * 0!$$

$$1! = 1 * _ _$$

Factorial of 4 (Recursive)

Calculate 4!

$$4! = 4 * (4-1)!$$

$$4! = 4 * 3!$$

$$4! = 4 * _ _ \text{Calculate 3!}$$

$$3! = 3 * (3-1)!$$

$$3! = 3 * 2!$$

$$3! = 3 * _ _ \text{Calculate 2!}$$

$$2! = 2 * (2-1)!$$

$$2! = 2 * 1!$$

$$2! = 2 * _ _ \text{Calculate 1!}$$

$$1! = 1 * (1-1)!$$

$$1! = 1 * 0!$$

$$1! = 1 * _ _$$

Calculate 0!

$$0! = 1$$

Factorial of 4 (Recursive)

Calculate 4!

$$4! = 4 * (4-1)!$$

$$4! = 4 * 3!$$

$$4! = 4 * _ _ \text{Calculate 3!}$$

$$3! = 3 * (3-1)!$$

$$3! = 3 * 2!$$

$$3! = 3 * _ _ \text{Calculate 2!}$$

$$2! = 2 * (2-1)!$$

$$2! = 2 * 1!$$

$$2! = 2 * _ _ \text{Calculate 1!}$$

$$1! = 1 * (1-1)!$$

$$1! = 1 * 0!$$

$$1! = 1 * 1$$

$$1! = 1$$

Factorial of 4 (Recursive)

Calculate 4!

$$4! = 4 * (4-1)!$$

$$4! = 4 * 3!$$

$$4! = 4 * _ _ \text{Calculate 3!}$$

$$3! = 3 * (3-1)!$$

$$3! = 3 * 2!$$

$$3! = 3 * _ _ \text{Calculate 2!}$$

$$2! = 2 * (2-1)!$$

$$2! = 2 * 1!$$

$$2! = 2 * 1$$

$$2! = 2$$

Factorial of 4 (Recursive)

Calculate 4!

$$4! = 4 * (4-1)!$$

$$4! = 4 * 3!$$

$$4! = 4 * _ _ \text{Calculate 3!}$$

$$3! = 3 * (3-1)!$$

$$3! = 3 * 2!$$

$$3! = 3 * 2$$

$$3! = 6$$

Factorial of 4 (Recursive)

Calculate 4!

$$4! = 4 * (4-1)!$$

$$4! = 4 * 3!$$

$$4! = 4 * 6$$

$$4! = 24$$

Recursive Factorial Function

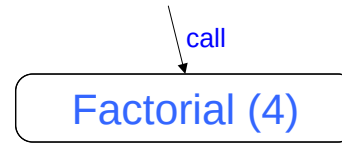
```
// Computes the factorial of a nonnegative integer.  
// Precondition: n must be greater than or equal to 0.  
// Postcondition: Returns the factorial of n; n is unchanged.  
int Factorial(int n)  
{  
    if (n == 0)  
        return (1);  
    else  
        return (n * Factorial(n-1));  
}
```

Tracing Factorial(4)

Factorial(4)

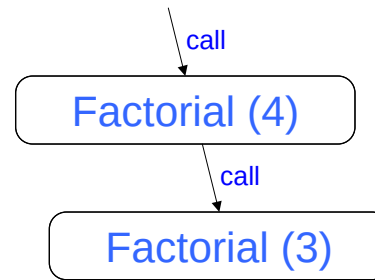
Tracing Factorial(4)

Factorial (4)



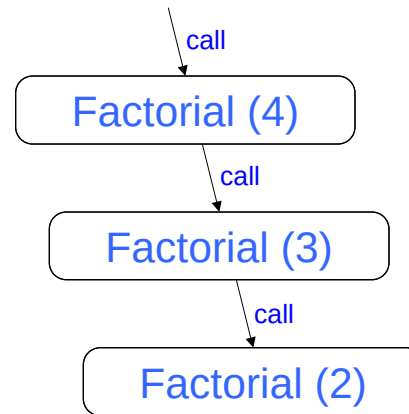
Tracing Factorial(4)

`Factorial(4)` `= 4 * Factorial(3)`



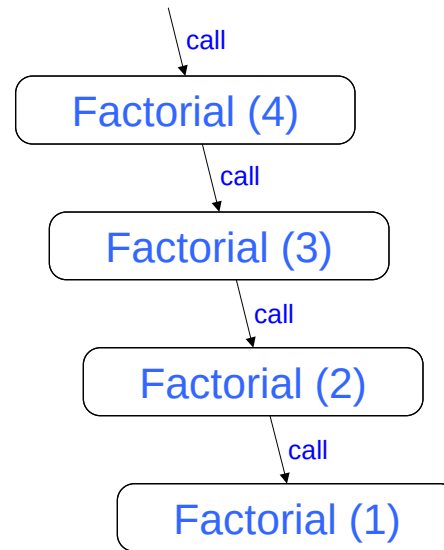
Tracing Factorial(4)

```
Factorial(4)      = 4 * Factorial(3)
                  = 4 * (3 * Factorial(2))
```



Tracing Factorial(4)

```
Factorial(4)      = 4 * Factorial(3)
                  = 4 * (3 * Factorial(2))
                  = 4 * (3 * (2 * Factorial(1)))
```



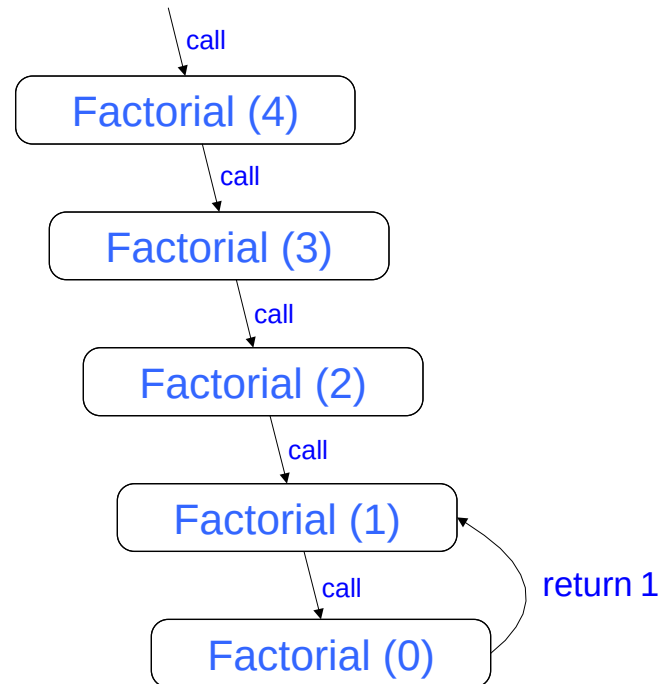
Tracing Factorial(4)

```
Factorial(4)      = 4 * Factorial(3)
                  = 4 * (3 * Factorial(2))
                  = 4 * (3 * (2 * Factorial(1)))
                  = 4 * (3 * (2 * (1 * Factorial(0))))
```



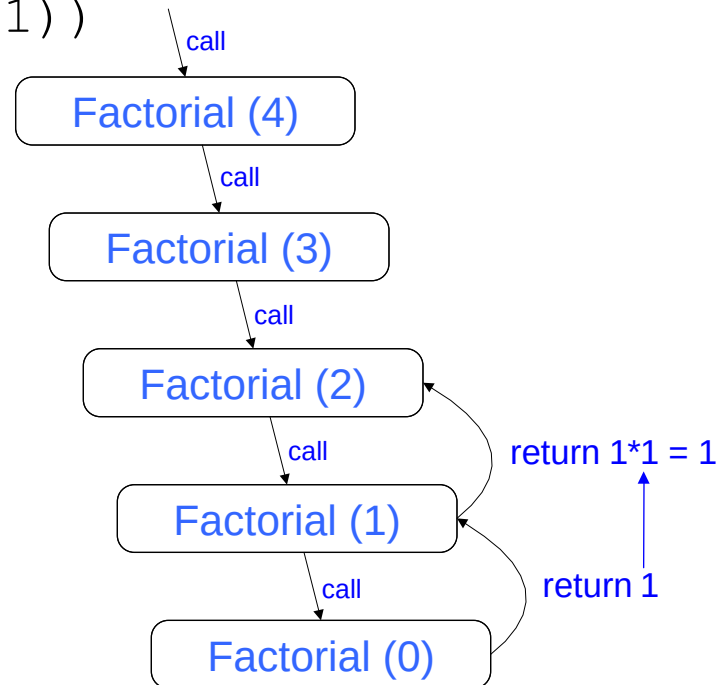
Tracing Factorial(4)

```
Factorial(4)      = 4 * Factorial(3)
                  = 4 * (3 * Factorial(2))
                  = 4 * (3 * (2 * Factorial(1)))
                  = 4 * (3 * (2 * (1 * Factorial(0))))
                  = 4 * (3 * (2 * (1 * 1)))
```



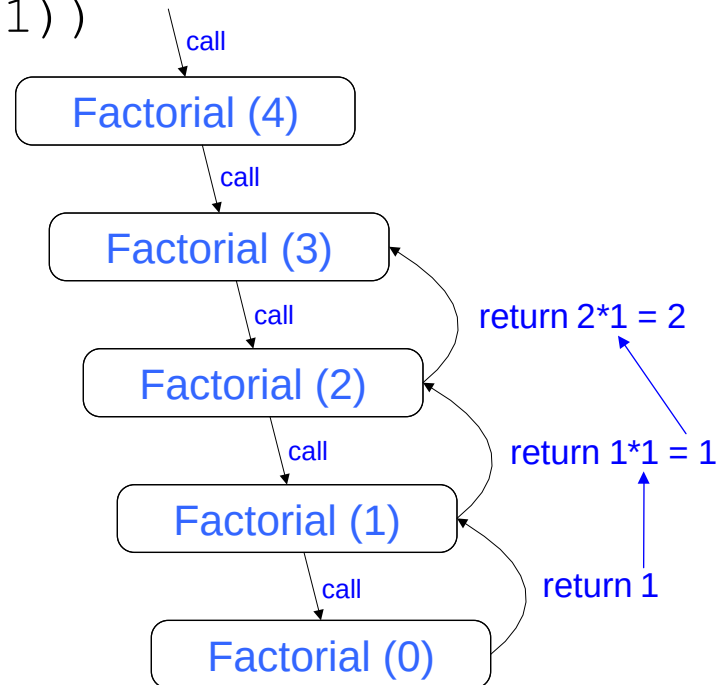
Tracing Factorial(4)

```
Factorial(4)      = 4 * Factorial(3)
                  = 4 * (3 * Factorial(2))
                  = 4 * (3 * (2 * Factorial(1)))
                  = 4 * (3 * (2 * (1 * Factorial(0))))
                  = 4 * (3 * (2 * (1 * 1)))
                  = 4 * (3 * (2 * 1))
```



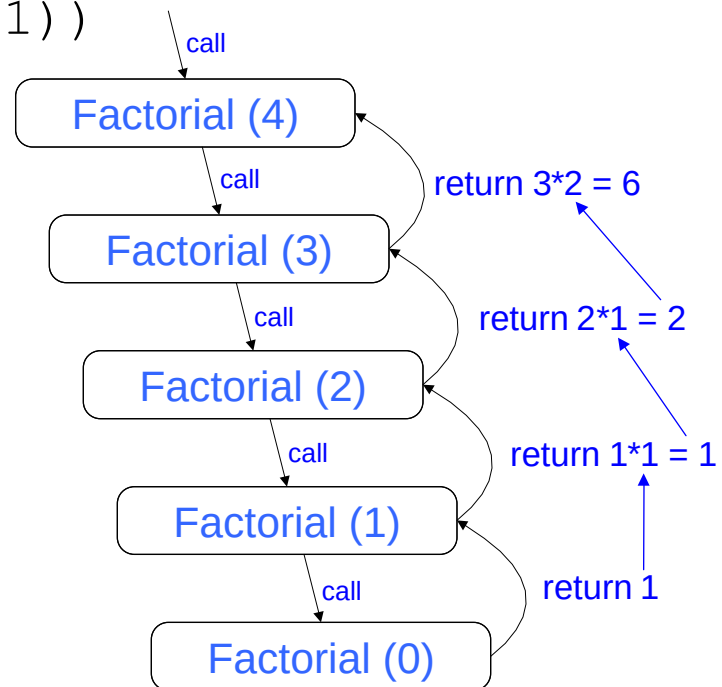
Tracing Factorial(4)

```
Factorial(4)      = 4 * Factorial(3)
                  = 4 * (3 * Factorial(2))
                  = 4 * (3 * (2 * Factorial(1)))
                  = 4 * (3 * (2 * (1 * Factorial(0))))
                  = 4 * (3 * (2 * (1 * 1)))
                  = 4 * (3 * (2 * 1))
                  = 4 * (3 * 2)
```



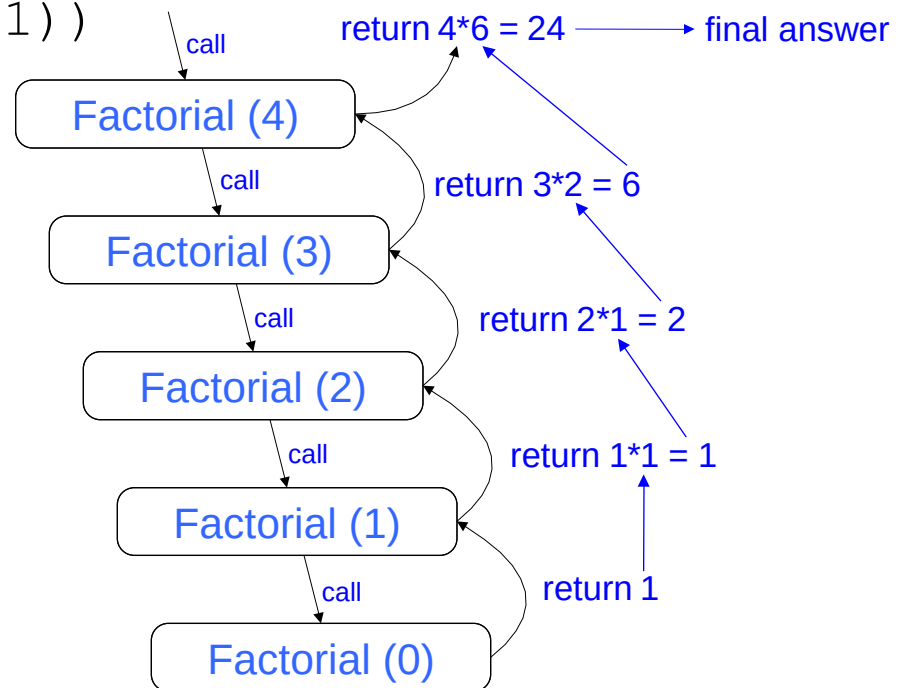
Tracing Factorial(4)

```
Factorial(4)          = 4 * Factorial(3)
                      = 4 * (3 * Factorial(2))
                      = 4 * (3 * (2 * Factorial(1)))
                      = 4 * (3 * (2 * (1 * Factorial(0))))
                      = 4 * (3 * (2 * (1 * 1)))
                      = 4 * (3 * (2 * 1))
                      = 4 * (3 * 2)
                      = 4 * 6
```



Tracing Factorial(4)

`Factorial(4)` `= 4 * Factorial(3)`
`= 4 * (3 * Factorial(2))`
`= 4 * (3 * (2 * Factorial(1)))`
`= 4 * (3 * (2 * (1 * Factorial(0))))`
`= 4 * (3 * (2 * (1 * 1)))`
`= 4 * (3 * (2 * 1))`
`= 4 * (3 * 2)`
`= 4 * 6`
`= 24`



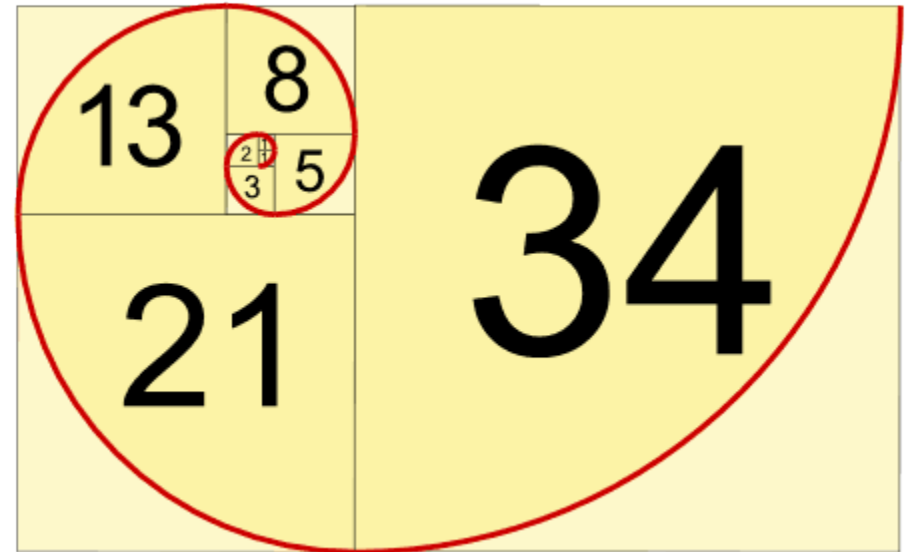
A Couple of Things You Should Know

- A **stack** is used to keep track of function calls.
- Whenever a new function is called, all its parameters and local variables are pushed onto the stack along with the memory address of the calling statement (this gives the computer the return point after execution of the function)

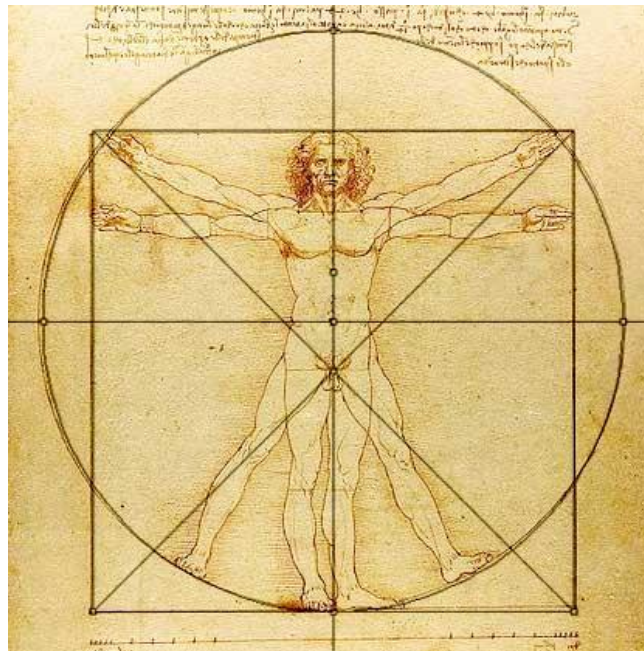
Pitfalls of Recursion

- If the recursion never reaches the base case, the recursive calls will continue until the computer runs out of memory and the program crashes. Experienced programmers try to examine the remains of a crash. The message “stack overflow error” or “heap storage exhaustion” indicates a possible runaway recursion.
- When programming recursively, you need to make sure that the algorithm is moving toward the base case. Each successive call of the algorithm must be solving a simpler version of the problem.
- Any recursive algorithm can be implemented iteratively, but sometimes only with great difficulty. However, a recursive solution will always run more slowly than an iterative one because of the overhead of opening and closing the recursive calls.

Fibonacci's Problem



“Fibonacci”
(Leonardo de Pisa)
1170-1240



Fibonacci Numbers

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

where each number is the sum of the preceding two.

- Recursive definition:
 - **$F(0) = 0;$**
 - **$F(1) = 1;$**
 - **$F(\text{number}) = F(\text{number}-1) + F(\text{number}-2);$**

Fibonacci Numbers

```
int Fibonacci(int n)
{
    if(n == 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return Fibonacci(n-1)+Fibonacci(n-2);
}
```

Fibonacci Numbers

```
int Fibonacci(int n)
{
    if(n == 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return Fibonacci(n-1)+Fibonacci(n-2);
}
```



Base case

Fibonacci Numbers

```
int Fibonacci(int n)
```

```
{
```

```
    if (n == 0)
```

```
        return 0;
```

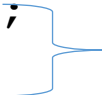
```
    else if (n == 1)
```

```
        return 1;
```

```
    else
```

```
        return Fibonacci(n-1)+Fibonacci(n-2);
```

```
}
```



**General
case**

Tracing Fibonacci(5)

